Kịch bản thuyết trình MLOps margin

Lời thoại cho 17 slide, khoảng 15 phút. Số trong ngoặc là thời gian gợi ý.

Mở đầu (2 phút)

Slide 1 · Bìa

> Em chào các anh chị. Em là Vũ Mạnh Cường, hôm nay em trình bày đề tài xây dựng nền tảng MLOps cho dự báo dữ liệu chứng khoán, do anh Nguyễn Ngọc Long hướng dẫn.

Slide 2 · Vấn đề

> Một model chạy tốt trong notebook chưa phải là một hệ thống. Khi đưa vào chạy thật, câu hỏi không còn là model chính xác đến đâu, mà là: dữ liệu mới có đến đúng giờ không, ai biết model đang chạy là phiên bản nào, khi dữ liệu đổi thì làm sao biết model đã kém đi, và khi cần thay model thì thay bằng cách nào mà không hỏng. Nếu không có MLOps, mọi thứ này làm bằng tay: chạy lại thủ công, sửa tay trên server, không ai kiểm soát phiên bản, và lỗi chỉ được phát hiện khi kết quả đã sai. Phần lớn công sức của một hệ thống ML nằm ở đây, không nằm ở thuật toán. Vì vậy đề tài này tập trung vào phần còn lại: tự động hóa dòng dữ liệu, quản lý phiên bản model, kiểm soát khi nào model mới được lên production, giám sát drift và hiệu quả thực tế, và triển khai có dấu vết. Bài toán margin chỉ là bối cảnh để chạy thử toàn bộ vòng đời đó.

Slide 3 · Ba bài toán, sáu model

> Trước tiên, loan_propensity là bài toán chính. Công ty chứng khoán cho khách hàng vay ký quỹ, gọi là margin, để mua cổ phiếu: khách chỉ phải bỏ một phần vốn, phần còn lại vay từ công ty. Bộ phận kinh doanh muốn biết khách nào sắp giải ngân khoản vay margin mới trong 30 ngày tới, để chào đúng người, đúng lúc. Model cho ra một điểm xác suất mỗi khách mỗi ngày, và danh sách điểm cao được đưa cho đội business. Câu hỏi tự nhiên là: sao không chỉ làm một bài toán này? Vì mục tiêu của đề tài là chứng minh một nền tảng dùng chung, không phải một model đơn lẻ. Nếu chỉ có một bài toán thì không biết hệ thống đủ tổng quát hay chỉ vừa khít với bài toán đó. Nên em chọn thêm hai bài toán, mỗi bài kiểm chứng một khía cạnh khác. Loan propensity kiểm chứng việc quản lý nhiều model của cùng một bài toán: tách ba model theo nhóm khách hàng nu, lu, au, vì ba nhóm có tỷ lệ nhãn dương khác nhau, khoảng 14%, 28% và 38%. Mỗi model có ngưỡng và AUC riêng, và cổng champion phải xét cả ba cùng lúc. Overdue risk, dự đoán khách có bắt đầu quá hạn trong 30 ngày, kiểm chứng việc đổi nhãn và đổi tập khách hàng được chấm, chỉ chấm nhóm đủ điều kiện, có 2 model. Campaign response, dự đoán khách có giải ngân sau chiến dịch trong 14 ngày, kiểm chứng đơn vị dữ liệu khác: mỗi dòng là một lần gửi chiến dịch chứ không phải khách hàng theo ngày, và chỉ cần 1 model chung. Kết quả là ba bài toán, sáu model, dùng chung nguồn Gold, chung 6 bảng đặc trưng và chung hạ tầng. Thêm một bài toán mới chỉ cần đổi cấu hình, không đổi hạ tầng. Các slide sau lấy loan propensity làm ví dụ.

Nền tảng (3 phút)

Slide 4 · Dòng dữ liệu

> Dữ liệu bắt đầu từ 8 bảng Gold: khách hàng, giao dịch, margin, giá, danh mục, chiến dịch. Từ đó em tính ra 6 nhóm đặc trưng, tính theo ngày, với cửa sổ 7 và 30 ngày. Nhãn được tách riêng. (Mô tả flow data theo luồng - ngắn gọn)

Slide 5 · Kiến trúc tổng quan

> Toàn bộ hệ thống chạy trên một cụm Kubernetes, và em chia cụm theo namespace, mỗi namespace một nhiệm vụ. Namespace airflow chứa Airflow, chỉ làm việc điều phối: lập lịch và nối thứ tự các task. Namespace pipeline là nơi làm việc thật: mỗi task Airflow tạo ra một pod ở đây bằng KubernetesPodOperator, pod chạy một lệnh Python rồi kết thúc. Tất cả pod dùng chung một image tên mlops-pipeline, để tính đặc trưng, huấn luyện, dự đoán, drift và đánh giá đều chạy cùng một môi trường. Namespace mlops chứa các dịch vụ chạy thường trực: MLflow để ghi run và đăng ký model, MinIO là kho lưu file với hai bucket, mlflow cho model và warehouse cho các bảng giám sát, PostgreSQL giữ metadata của MLflow và catalog của các bảng Iceberg, và deployer đọc alias production để cập nhật manifest triển khai. Namespace argocd chứa ArgoCD, tự đồng bộ cấu hình từ Git vào cụm. Cuối cùng, namespace kserve và serving chạy KServe, phục vụ model thành endpoint. Nhờ tách như vậy, thành phần chạy theo lịch, thành phần chạy thường trực và thành phần phục vụ không lẫn vào nhau, dễ theo dõi và dễ sửa riêng từng phần. Trên hạ tầng này, hệ thống chia thành năm luồng. Một, đưa dữ liệu Gold vào Feature Store. Hai, huấn luyện và đưa model lên production. Ba, dự đoán batch hằng ngày kèm giám sát drift. Bốn, triển khai model lên KServe bằng GitOps. Năm, đánh giá hiệu quả thực tế. Vòng huấn luyện lại nối luồng ba và năm trở về luồng hai.

Slide 6 · Lịch vận hành trong tuần

> Em trình bày logic vận hành qua một ngày điển hình. Khoảng 6 giờ sáng giờ Việt Nam, khi dữ liệu Gold mới của ngày hôm qua đã có, Airflow chạy DAG feature_store_daily. DAG này tính 6 nhóm đặc trưng từ Gold và ghi vào các bảng fs, mỗi ngày một partition. Chỉ khi dữ liệu qua cổng chất lượng, ngày đó mới được đánh dấu hoàn tất. Đến 7 giờ, DAG dự đoán batch chạy, dựa trên các bảng fs mới vừa có. Nó chấm điểm toàn bộ khách hàng bằng model đang ở production, để xem khách nào có khả năng vay margin cao. Kết quả là bảng điểm theo ngày. Từ bảng điểm đó, đội business lấy danh sách khách điểm cao để đưa ra offer phù hợp. Cùng DAG này còn chạy bước drift: so dữ liệu hôm nay với dữ liệu model đã học, để biết model có đang lạc hậu không. Mỗi Chủ Nhật 7 giờ sáng, hai DAG chạy theo tuần: huấn luyện, để thử một model mới trên dữ liệu mới hơn, và đánh giá, để so điểm đã dự đoán với nhãn thật khi nhãn đã chín. Ngoài lịch tuần, huấn luyện còn được gọi khi drift ở mức critical. Chạy liên tục là deployer: cứ 60 giây nó kiểm tra alias production. Nếu người vận hành vừa duyệt một model mới, deployer cập nhật manifest, ArgoCD đồng bộ, và KServe chạy model mới. Thứ tự quan trọng: đặc trưng xong lúc 6 giờ thì dự đoán mới chạy lúc 7 giờ, và nếu đặc trưng không qua kiểm tra chất lượng thì dự đoán không chạy.

Năm luồng (8 phút)

Slide 7 · Luồng 1: Gold sang Feature Store

> Đây là luồng đầu tiên: biến dữ liệu Gold thành bảng đặc trưng. DAG feature_store_daily chạy 6 giờ sáng, gồm năm task nối tiếp. Task một, check_source: kiểm tra nguồn Gold đã đủ ngày T chưa và đúng schema chưa. Nếu chưa thì task lỗi và Airflow thử lại, không tính gì, để tránh ghi partition rỗng hoặc sai. Task hai, compute_features: dùng DuckDB đọc Gold ngày T cùng lịch sử lùi tối đa 30 ngày, tính 6 nhóm đặc trưng với cửa sổ 7 và 30 ngày, rồi ghi ra một bảng tạm. Em dùng DuckDB vì phép biến đổi này là các phép gộp theo cửa sổ ngày trên dữ liệu dạng bảng, DuckDB chạy ngay trong pod, không cần dựng thêm một cụm xử lý riêng, và mỗi lần chỉ cần 30 ngày lịch sử nên rất nhẹ. Task ba, write_partitions: ghi vào 6 bảng Iceberg trên MinIO bằng overwrite có điều kiện report_date bằng T. Chưa có partition thì tạo, có rồi thì thay. Em chọn Iceberg vì nó chia dữ liệu theo partition, nên ghi hay thay một ngày không đụng tới ngày khác, và chạy lại ngày T luôn an toàn, gọi là idempotent. Task bốn, quality_gate: kiểm tra trên partition T, với ngưỡng cứng là không được có dòng vi phạm. Kiểm tra đúng kiểu dữ liệu theo hợp đồng schema, khóa master_account và report_date không null và không trùng, 24 cột đặc trưng không null, và giá trị hợp lệ. Không đạt thì task đỏ. Task năm, publish_marker: ghi một dòng hoàn tất vào bảng điều khiển control.fs_status. DAG dự đoán dùng một cảm biến chờ dòng này, nên dữ liệu không qua cổng thì dự đoán không chạy. Ngoài ra mỗi tuần có một task bảo trì: xóa partition quá 12 tháng, hết hạn snapshot cũ và dọn file mồ côi để dung lượng không phình ra.

Slide 8 · Luồng 2: Chia dữ liệu theo thời gian

> Ở luồng huấn luyện, em chia dữ liệu theo thời gian chứ không chia ngẫu nhiên. Train từ 01/02 đến 30/06, validation từ 31/07 đến 30/09, test từ 31/10 đến 31/12 năm 2025. Giữa các đoạn có embargo 30 ngày để nhãn của đoạn trước không rò sang đoạn sau. Train để học tham số và tính trọng số lớp cho từng segment. Validation để chọn giữa LR và LightGBM theo ROC-AUC, và chọn ngưỡng max-F1. Test chỉ chấm đúng một lần, làm đầu vào cho cổng champion.

Slide 9 · Cổng champion / challenger

> Đây là luồng huấn luyện, DAG loan_propensity_train, gồm hai task: train rồi report_gate. Nó chạy Chủ Nhật 7 giờ sáng, hoặc được DAG dự đoán gọi khi drift critical, hoặc người vận hành bấm Trigger. Bước một, Airflow tạo pod train ở namespace pipeline. Pod gắn volume pipeline-data, đọc run đặc trưng đã duyệt, và dừng ngay nếu báo cáo chất lượng không phải PASS, tức không huấn luyện trên dữ liệu chưa đạt. Bước hai, đọc training_dataset, lọc theo bài toán và tách theo segment nu, lu, au, rồi chia train, validation, test theo thời gian như slide trước. Bước ba, với mỗi segment huấn luyện hai loại model: Logistic Regression và LightGBM, dùng trọng số lớp để bù lớp dương hiếm. Chọn model có ROC-AUC trên validation cao hơn, và chọn ngưỡng quyết định là điểm có F1 lớn nhất trên validation. Tập test chỉ chấm đúng một lần, sau khi mọi lựa chọn đã chốt. Bước bốn, ghi vào MLflow: mỗi segment một run, lưu tham số, 30 metric và file model. Model được đăng ký tên theo dạng bài toán gạch dưới segment, version mới nhận alias staging cùng các tag như threshold, test_roc_auc, data_end. File model nằm ở MinIO, metadata nằm ở PostgreSQL. Bước năm, cổng champion challenger, như đã nói: so version staging với version production trên cùng các dòng test, và phải đạt cả ba segment. Kết quả ghi vào tag promotion_gate là PASS, BASELINE_PASS hoặc FAIL. Bước sáu, task report_gate chỉ in ra kế hoạch chuyển production, không thay đổi gì. Nếu bị từ chối thì task đỏ, đó là kết luận chưa nên chuyển, không phải lỗi. Bước bảy, người vận hành xem metric rồi duyệt, ưu tiên băng lệnh promote với cờ approve. Lệnh này từ chối nếu ba segment không cùng một lần huấn luyện hoặc cổng chưa đạt, và ghi lại ai duyệt, lúc nào, thay version nào. Airflow không bao giờ tự đặt alias production. Điều kiện đạt của cổng: AUC không thấp hơn lựa chọn lớn hơn giữa AUC production trừ 0,03 và sàn 0,75, và Recall không thấp hơn Recall production trừ 0,05. Sàn 0,75 ngăn model tụt dần qua nhiều lần huấn luyện. Cổng là all-or-nothing, một segment trượt thì cả bài toán trượt.

Slide 10 · MLflow: từ staging lên production

> Đây là ví dụ thật: model loan_propensity_au version 11, cổng PASS, có tag người duyệt và data_end. Người vận hành vào MLflow đặt alias production cho version này. Nếu có sự cố, rollback chỉ là trỏ alias production về version cũ. Version cũ không bị xoá.

Slide 11 · Luồng 3: Dự đoán batch hăng ngày

> Đây là DAG loan_propensity_predict, chạy 7 giờ sáng hăng ngày, gồm bốn task nối tiếp là predict, drift, retrain_check, trigger_retrain. Mỗi task là một pod mới ở namespace pipeline, tạo bởi Airflow, dùng image mlops-pipeline. Slide này nói về task predict. Bước một, chốt version production. Pod predict gọi MLflow, chạy trong namespace mlops, để đọc alias production của ba model loan_propensity au, lu, nu. Pod đổi từng alias thành một số version cố định, ví dụ version 11, rồi nạp đúng version đó từ MLflow. Em làm vậy để nếu người vận hành đổi alias giữa lúc đang chạy, một lần chạy không bị trộn hai version. Segment nào chưa có production thì không được chấm. Lưu ý dự đoán batch nạp model trực tiếp từ MLflow, không đi qua KServe. Bước hai, đọc đặc trưng. Pod đọc 6 bảng fs trên Iceberg, nằm ở MinIO, đúng partition report_date bằng ngày chấm, rồi ghép theo master_account và report_date thành một dòng đủ đặc trưng cho mỗi tài khoản. Bước ba, chấm điểm. Mỗi khách hàng được chấm bằng model của đúng segment mình, lấy xác suất lớp dương làm điểm. Pod chưa áp ngưỡng ở bước này, mà lưu điểm thô để dùng cho nhiều mục đích như drift, đánh giá, hay đội business tự chọn ngưỡng. Bước bốn, ghi kết quả vào bảng Iceberg monitoring.pred_loan_propensity. Catalog của bảng nằm ở PostgreSQL, dữ liệu parquet nằm ở MinIO bucket warehouse, partition theo score_date. Ghi bằng overwrite có điều kiện, nên chạy lại cùng ngày thì chỉ thay đúng partition đó. Các cột gồm khách hàng, ngày chấm, segment, điểm, tên và version model, feature_run_id và thời điểm chấm. Ví dụ ngày 30/01/2026: 4.731 dòng được chấm, gồm au 1.807, lu 1.965, nu 959, tất cả bằng version 11. Chấm xong thì task drift chạy, so đặc trưng với ngày tham chiếu, rồi retrain_check quyết định có gọi huấn luyện lại không. Phần đó em nói ở slide sau.

Slide 12 · Giám sát drift và quyết định retrain

> Sau khi chấm điểm, Evidently so 24 đặc trưng của ngày hôm nay với ngày data_end của version production. Drift ratio là số cột lệch chia 24, chia thành bốn mức: none, medium khi tối đa 0,20, high khi tối đa 0,50, và critical khi trên 0,50. Chỉ tự gọi huấn luyện lại khi đủ ba điều kiện: drift critical, có nhãn mới hơn data_end, và đã qua cooldown 7 ngày. Lý do đủ ba: drift cao mà chưa có nhãn mới thì huấn luyện lại cũng không học thêm gì; cooldown tránh huấn luyện liên tục. Ví dụ 30/01/2026 so với 31/12/2025: 12 trên 24 cột lệch, 50%, mức high, nên chưa tự retrain.

Slide 13 · Luồng 4: Triển khai bằng GitOps

> Khi alias production đổi, deployer, chạy mỗi 60 giây, sẽ phát hiện. Nó lập kế hoạch: bỏ qua, không đổi, bị chặn, tạo mới hay cập nhật. Rồi commit manifest lên GitHub, ArgoCD tự đồng bộ vào cụm, KServe tạo pod mới và gỡ pod cũ khi pod mới sẵn sàng. Ý chính: mọi thay đổi đều có commit. Nếu ai sửa tay trong cụm, ArgoCD sẽ hoàn nguyên nhờ self-heal.

Slide 14 · Kết quả triển khai

> Kết quả thực tế: khoảng 23 giây từ commit đến khi ArgoCD đồng bộ, thêm khoảng 17 giây để pod mới Ready. Application serving ở trạng thái Healthy và Synced. Bước predict_check so điểm của endpoint với model trong MLflow: version 11, PASS, chênh lệch 0,0. Dự đoán batch nạp model trực tiếp từ MLflow; KServe phục vụ các lời gọi lẻ.

Slide 15 · Luồng 5: Đánh giá hiệu quả thực tế

> Drift chỉ là dấu hiệu gián tiếp. Để biết model còn tốt không, em ghép điểm với nhãn thật khi nhãn đã chín, sau 30 ngày. Cửa sổ 25 đến 31/12/2025 có 33.117 dòng. AUC thực tế: au 0,850, lu 0,848, nu 0,908, đều cao hơn AUC lúc test và đều mức OK. Quy tắc: AUC tụt hơn 0,05 là WARN, tụt hơn 0,10 là RETRAIN.

Slide 16 · Vòng huấn luyện lại

> Huấn luyện lại có ba đường vào. Một là lịch tuần, Airflow tự chạy. Hai là drift critical, tự gọi khi có nhãn mới và hết cooldown 7 ngày. Ba là AUC thực tế tụt, WARN kéo dài hoặc RETRAIN, lúc này người vận hành chạy tay. Dù đi đường nào, model mới cũng phải qua cổng champion và người duyệt mới lên production. Vì vậy vòng lặp tự động nhưng không tự ý thay model.

Kết (1 phút)

Slide 17 · Tổng kết

> Tóm lại, em xây một nền tảng MLOps chạy trọn vòng đời cho 3 bài toán, 6 model, với ba điểm chính. Thứ nhất, tự động: năm luồng Airflow, ghi theo partition, chạy lại an toàn. Thứ hai, có kiểm soát: model có phiên bản, qua cổng champion và người duyệt, triển khai GitOps. Thứ ba, có giám sát: drift hằng ngày, AUC thực tế hằng tuần, retrain khi đủ điều kiện. Em xin cảm ơn và sẵn sàng nhận câu hỏi.

Gợi ý trả lời câu hỏi

- **Vì sao sàn AUC 0,75?** Chặn model tụt dần: nếu chỉ so với production trừ 0,03 thì mỗi lần thay có thể kém thêm một chút. Sàn cố định giữ mức tối thiểu tuyệt đối.
- **Vì sao cooldown 7 ngày?** Tránh huấn luyện liên tục khi drift dao động; trùng chu kỳ lịch huấn luyện tuần.
- **Vì sao cần nhãn mới hơn data_end?** Không có nhãn mới thì huấn luyện lại dùng cùng dữ liệu cũ, model không học thêm gì.
- **Vì sao chia theo thời gian, có embargo?** Nhãn nhìn 30 ngày về tương lai; embargo 30 ngày chặn rò nhãn giữa các đoạn.
- **Vì sao người duyệt mới được lên production?** Cổng tự động kiểm chỉ số, người duyệt kiểm bối cảnh nghiệp vụ; Airflow chỉ được đặt staging.
- **Nếu model mới tệ hơn thì sao?** Rollback bằng cách trỏ alias production về version cũ; version cũ không bị xoá.
- **Drift 50% mà sao không retrain?** 12/24 cột là mức high; critical cần trên 50%, và còn phải có nhãn mới và hết cooldown.